

Between Text and Events: Process Context Graph (ProCG)

MOHAMMAD DALLASH, ClearSky AI, United States

WALEED ALZAMIL, ClearSky AI, United States

KYRILLOS ESKANDER, ClearSky AI, United States

XIXI LU, Utrecht University, The Netherlands

AHMED ELKHOLY, ClearSky AI, United States

Process mining relies primarily on structured event data, although substantial process-relevant context may exist only in unstructured sources such as emails, reports, and notes. Recent advances in large language models (LLMs) make it possible to extract this information and incorporate it into process analysis. Directly mapping text to event-log structures, however, may discard important context, including actor identities, decisions, evidence, and rationales. We introduce the Process Context Graph (ProCG), a knowledge-graph-based intermediate representation that connects information extracted from unstructured sources with structured event data while preserving contextual relationships. We define its conceptual model and present an LLM-based pipeline for constructing ProCGs from unstructured text. We evaluate the approach in a controlled activity-sequence reconstruction setting based on the BPI Challenge 2017 event log, in which sequences of one to five event occurrences are removed from the event log, and synthetic emails are generated to describe the corresponding process behavior. Using the local ProCG of the process instance instead of the synthetic email improves the exact ordered match by 2.8 percentage points (0.740 to 0.768). Enriching the local ProCG with related context retrieved from other process instances improves the exact ordered match by a further 4.8 percentage points, from 0.768 to 0.816, and reaches 0.821 when the enriched ProCG is combined with the email. Compared to the synthetic-email baseline, multiset F1 improves by 3.4 percentage points (0.914 to 0.948). These results indicate that explicitly structuring contextual information can support reconstruction of activity sequences missing from structured event data.

Additional Key Words and Phrases: process mining, object-centric event data, large language models, knowledge graphs, unstructured data, event data reconstruction

1 Introduction

Process mining relies primarily on structured event data that describe what happened, when it happened, and which cases, objects, or resources were involved. The Process-relevant context about activities, decisions, and interactions may instead reside only in unstructured sources such as emails, reports, and notes, leaving it unavailable to conventional process mining analysis.

Recent advances in large language models (LLMs) enable process information to be extracted from natural language and transformed into event-log structures [3]. However, direct mapping to event records can collapse contextual relationships between actors, decisions, evidence, and rationales, as well as their provenance.

We introduce the Process Context Graph (ProCG), a knowledge-graph-based intermediate representation that connects unstructured information with structured event data. ProCG represents actors, evidence, decisions, and rationales as first-class entities, preserving their relationships and provenance without requiring activity labels at construction time.

We also present an LLM-based construction pipeline that transforms each source into an ontology-conformant subgraph and merges it through deterministic identity resolution. The resulting graph supports local context retrieval and cross-instance retrieval of semantically and structurally similar decisions and rationales see Figure 4.

We evaluated ProCG in an activity-sequence reconstruction task using the BPI Challenge 2017 event log. We remove event occurrences corresponding to activity sequences of lengths one to five from application traces, generate synthetic emails describing the removed behavior, and compare reconstruction using the remaining event data, the email, the ProCG, and cross-instance enrichment. The comparison tests whether structuring textual information as a ProCG improves reconstruction over email alone and whether retrieved and enriched context provides additional benefit.

Our work contributions: (i) a conceptual model for representing process-relevant context extracted from unstructured sources and linking it to structured event data; (ii) an LLM-based construction pipeline for building ProCGs while preserving provenance and enforcing a constrained graph schema; (iii) a retrieval and enrichment mechanism that combines semantic and structural similarity to identify related decisions and rationales across process instances; and (iv) a controlled evaluation of the representation on the reconstruction of missing activity sequences.

Section 2 reviews related work. Section 3 presents ProCG and its construction and retrieval mechanisms. Section 4 describes the experimental design, Section 5 reports the results, and Section 6 concludes the paper.

2 Background and Related Work

LLMs and unstructured data in process mining. LLMs are increasingly used in process mining to reason over process artifacts and support process-related tasks [1, 2, 12]. Dani et al. [6] use LLMs to generate event-data extraction queries from relational databases and show that the domain and the data context affect the extraction quality. Their setting assumes that the relevant process information is already available in structured information systems.

Buss et al. [3] address a setting closer to ours by constructing object-centric event logs from textual descriptions using heuristic NLP and generative AI to identify event occurrences, objects, attributes, and their relationships. Their results demonstrate the potential of generative models for recovering object-centric process information from text.

These approaches primarily map extracted information directly to event-log structures. Contextual relationships among actors, decisions, evidence, and rationales may therefore remain implicit or disconnected from existing event data.

Graph-based representations of process context. Graph representations provide a natural way to preserve relational information. In process mining, Event Knowledge Graphs (EKGs), for example, represent events together with the entities to which they relate, enabling analysis beyond a single case notion [7]. Subsequent work uses EKGs for graph-based querying and aggregation of task executions, actors, and interactions across behavioral dimensions [10]. These approaches primarily convert already structured event data into EKGs and do not address contextual knowledge extracted from unstructured sources.

ProCG connects these two directions by linking information extracted from unstructured sources [14] with existing process event data. Rather than transforming text directly into event records, it represents contextual entities and relations such as actors, interactions, decisions, evidence, and rationales. We construct ProCGs from OCEs and emails using LLMs and evaluate whether this contextual layer improves the reconstruction and enrichment of incomplete event data.

3 Process Context Graph (ProCG)

A *Process Context Graph* (ProCG) serves as an intermediate representation between unstructured sources and event logs. It preserves contextual information extracted from unstructured data in an event-aware graph. Unstructured

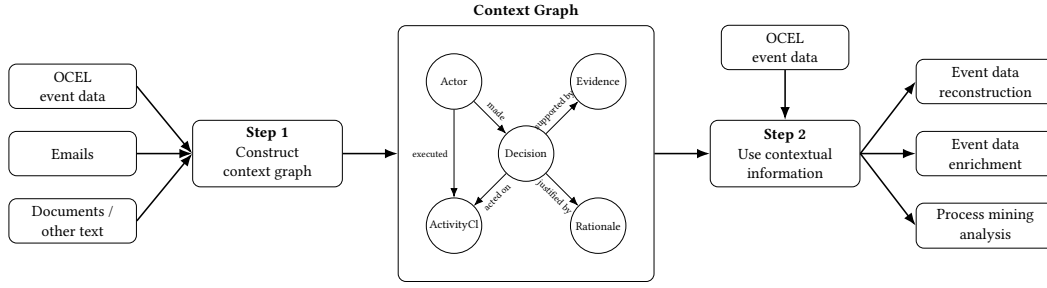


Fig. 1. The construction of a context graph and the use of it.

sources often describe relationships between activities, actors, decisions, supporting evidence, and rationales rather than isolated events.

Where available, ProCG represents actors, decisions, evidence, and rationales as first-class entities connected by the typed relations defined in Sect. 3.1.

3.1 Conceptualization

The ProCG follows a knowledge-graph-based representation in which nodes denote process-relevant entities and edges denote relations between them. We distinguish six entity types:

- An *Actor* is a person or team involved in executing events of the process.
- A *ProcessInstance* is one case execution of the process.
- An *ActivityCluster* is an unnamed placeholder for the activities a source reports. It carries no label: recovering which activities occurred is the downstream task.
- A *DecisionNode* represents a decision expressed in the unstructured context and links it to the relevant Actor node.
- An *EvidenceNode* is a verbatim span of the source that states a fact, together with its provenance.
- A *RationaleNode* is the reason given for a decision, supported by one or more EvidenceNodes.

These elements preserve contextual relationships that would otherwise be lost. For example, a decision can be linked to the actor who made it, the relevant ActivityCluster, its supporting evidence, and its rationale.

These entities are connected by the closed set of typed relations in Table 1. The builder writes no other relation types, regardless of extractor output. Execution verbs are stored as named edges from an actor to the cluster; they are never activity labels. The six verbs are collapsed to a single `linked_to` edge only when a subgraph is rendered for reconstruction Sect. 3.3.3.

The relation set is intentionally closed. `part_of` anchors each unnamed ActivityCluster in its ProcessInstance, while the execution relations connect actors to that cluster without naming the underlying activity, which remains the downstream prediction task. `produced`, `produced_evidence`, `mentions`, and `attached_to` to preserve provenance by recording what evidence belongs to the case’s work, who produced it, whom it refers to, and how related artifacts such as attachments are linked. `has_decision`, `acted_on`, `made_decision`, `supported_by`, `justified_by`, and `explained_by` capture what decision was made, who made it, what part of the case it concerns, which evidence supports it, and the rationale behind it. This schema preserves the intended context structure while preventing arbitrary edges or activity labels from entering the graph.

Table 1. Allowed relations of the ProCG. The six execution verbs are alternative labels of the same Actor→ActivityCluster pattern.

Relation	From	To
part_of	ActivityCluster	ProcessInstance
prepared, reviewed, ran, completed, approved, signed_off	Actor	ActivityCluster
produced	ActivityCluster	EvidenceNode
produced_evidence	Actor	EvidenceNode
has_decision	ActivityCluster	DecisionNode
acted_on	DecisionNode	ActivityCluster
made_decision	Actor	DecisionNode
justified_by	DecisionNode	RationaleNode
supported_by	DecisionNode	EvidenceNode
explained_by	EvidenceNode	RationaleNode
mentions	EvidenceNode	Actor
attached_to	EvidenceNode	EvidenceNode

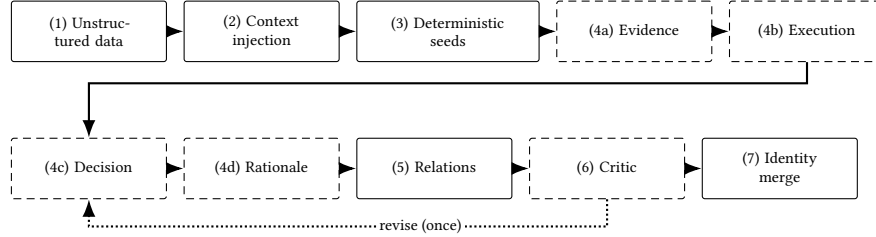


Fig. 2. Construction of a ProCG subgraph from one incoming source of unstructured data. Solid boxes are deterministic; dashed boxes invoke an LLM. Stage 6 may revise once at the decision classifier.

3.2 Constructing ProCGs

Construction proceeds one source at a time. Each incoming unstructured source is converted into an ontology-conformant subgraph and merged into the ProCG associated with the corresponding log Fig. 1. The procedure is incremental, so independently constructed subgraphs do not require later reconciliation. Our extraction method processes one source (e.g., an email) at a time.

The pipeline comprises seven stages.

- (1) *Unstructured incoming source of data.* Each unstructured source arrives with participant and timestamp metadata and is already associated with the relevant process instance and actors. If the source cannot be linked to a process instance, it does not contribute to the ProCG or generate new nodes.
- (2) *Context injection.* Observed OCEL events from the same case, before and after the unstructured source, are prepended as prompt context so the extractors can interpret the source against the case history. These context lines never enter the graph.
- (3) *Deterministic seeds.* Before interpreting the unstructured data, the pipeline deterministically creates an Actor for each linked participant, a ProcessInstance for the case, and one unnamed ActivityCluster attached to it, using only available metadata. No activity name is introduced.
- (4) *Contextual extraction.* The next four steps in Fig. 2 interpret the unstructured source.

From: christopher_jackson
Subject: Update on Application_944269054

Hi manager,
Regarding case Application_944269054, the application has been closed with no loan approval. This means we won't be moving forward on our side.
Best,
Christopher

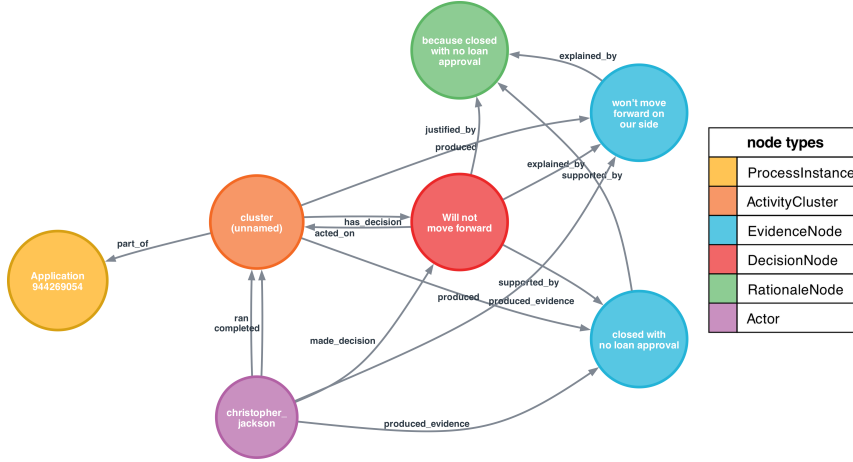


Fig. 3. Incoming email and the resulting ProCG subgraph. Evidence, the reject decision, and its rationale hang off the unnamed ActivityCluster; the process instance is the case and the actor is the sender.

- *Evidence* (Step 4a) splits the verbatim source text into fact-level spans, yielding one EvidenceNode per citable claim.
- *Execution* (Step 4b) records who acted on the cluster, using the six verbs of Table 1. Because the cluster is unnamed, this step never has to say *which* activity an actor acted on.
- *Decision* (Step 4c) records committed choices, and emits nothing when the source contains none.
- *Rationale* (Step 4d) records the justification associated with each decision, again from the source text.

Figure 3 shows an incoming email and the subgraph constructed from it. Two evidence spans, a reject decision, and a rationale hang off the unnamed ActivityCluster; no activity label such as A_Denied is written into the graph.

- (5) *Relation assembly*. A deterministic builder writes only the edges of Table 1: the cluster to its case, actors to the cluster, evidence to its producer and to the objects it mentions, and decisions to the evidence and rationale that support them.
- (6) *Critic*. Deterministic checks confirm that the subgraph respects the ontology (unnamed cluster, no quotes from injected context, decisions grounded in evidence). An LLM critic then reviews the subgraph; findings trigger at most one revision round, which re-enters at the decision classifier.
- (7) *Identity merge*. The subgraph is merged into the shared graph by identity: process instances by case identifier, actors by canonical identifier, and evidence, decisions, and rationales as immutable facts of the source message.

After this persistence step, decision and rationale nodes receive embeddings used at query time, which we return to in Sect. 3.3.

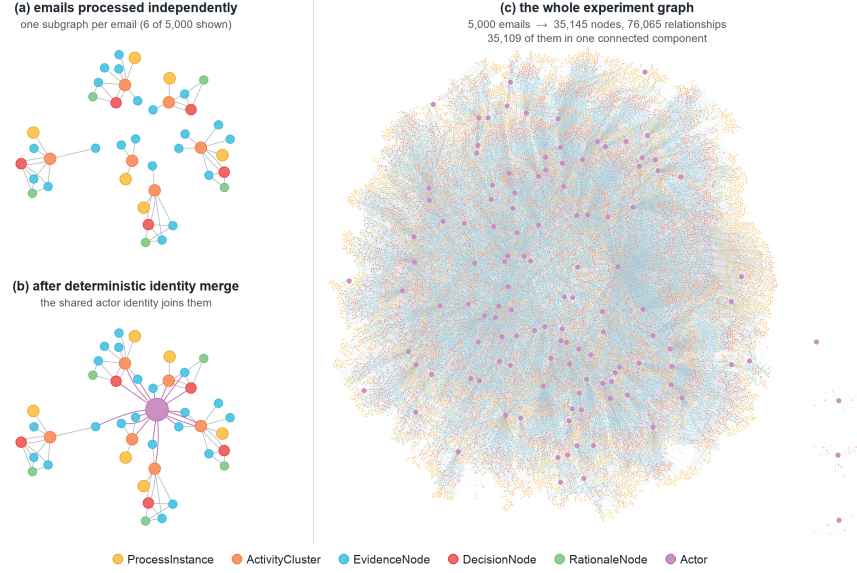


Fig. 4. The ProCG of the whole experiment corpus. Emails are processed independently, each producing its own subgraph (a, six of the 5,000 shown). The identity merge of step 7 joins subgraphs that name the same actor, without similarity matching (b, same node positions as (a)). At corpus scale this yields one graph (c): 5,000 emails, 35,145 nodes and 76,065 relationships, of which 35,109 lie in a single connected component. The 122 actor nodes (purple) are the only joins between per-email subgraphs.

Merged graph structure. Steps 1–6 process each email independently, so ingestion is order-independent and each email yields its own subgraph. Step 7 merges these subgraphs by identity alone, never by similarity or by an LLM judgment. Fig. 4 shows the result for the 5,000-email corpus of Sect. 4: 35,145 nodes and 76,065 relationships. Every email in that corpus concerns a distinct process instance, so the joins that actually connect subgraphs are the 122 actor identities. Deleting them splits the graph back into exactly 5,000 components, one per email, whereas as stored 35,109 of the 35,145 nodes lie in a single connected component. Cross-case retrieval in Sect. 3.3 traverses these joins.

3.3 Querying and Retrieval of the Process Context Graph

After construction, the ProCG contains the structure extracted from all processed sources. A consumer does not need the full graph; *retrieval* selects the relevant portion and serializes it to text.

Retrieval operates on an *anchor* node. Starting from that node, it collects the surrounding graph and renders the resulting *subgraph* as text. Consumers therefore receive a sequence of subgraphs rather than the full graph.

Retrieval begins with the process instance currently under consideration. From this initial subgraph, the consumer can query a decision or rationale for similar nodes elsewhere in the graph Sect. 3.3.2 and open the process instance associated with any returned node Sect. 3.3.3. Each step returns additional subgraphs and identifiers that can be used in subsequent steps. In our setting, the consumer is an LLM that decides after each round whether to continue. Sect. 3.3.3 describes the rendering format.

3.3.1 The first anchor. Retrieval first opens the ProcessInstance under consideration and collects the graph within two hops: the activity cluster, attached actors, decisions, evidence spans quoted from the unstructured data, rationales, and the relations among them, as illustrated in Fig. 3.

The traversal used to collect a subgraph never passes through an actor into another process, so each subgraph describes one process instance. Because actor nodes are merged across the full graph, allowing traversal through them would quickly reach unrelated processes handled by the same person.

3.3.2 Querying a decision or a rationale. Given a decision or a rationale in the subgraph it holds, the consumer can ask for the decisions or rationales most similar to it elsewhere in the graph. Only these two node types can be queried this way: they carry what was committed to and why, whereas matching on evidence would retrieve wording rather than meaning. Decisions are matched against decisions and rationales against rationales, so a result is always comparable to the node queried. Retrieval takes the top- k candidates and expands *all* of them, returning each candidate’s score, text, process identifier, and subgraph.

Similarity combines semantic and structural components. The *semantic* vector t (we use openai text-embedding-3-large) embeds the node text, capturing what was decided or why. The *structural* vector s (using FastRP [5]) over the full graph, captures the node’s position among actors, evidence, and clusters regardless of wording. Both embeddings are computed once after construction. A query q ranks candidates c of the same class by

$$\text{score}(q, c) = \alpha \cdot \cos(t_q, t_c) + (1 - \alpha) \cdot \cos(s_q, s_c), \quad (1)$$

where $\alpha \in [0, 1]$ controls the mixture; we use $\alpha = 0.5$. Candidates from a process already held by the consumer are excluded, so every result refers to a previously unseen process.

3.3.3 Rendering a subgraph. Every result also names the process it belongs to, which the structure around its node identifies unambiguously. The consumer can then open that process and receive it as a subgraph of its own.

Fatemi et al. [8] show that a graph can be handed to a language model as plain text, and that how it is written out measurably changes how well the model reasons over it. We adopt this approach and use one encoding for all subgraphs, regardless of how they were retrieved. A subgraph is rendered as a list of nodes followed by a list of relations Fig. 5. Each node gets a short tag from its type and a counter (E evidence, AC activity cluster, P process instance, D decision, R rationale, A actor). A tag is shorthand for writing the relations, not an identifier: the numbering starts again in each subgraph, so E1 in one subgraph and E1 in another are unrelated. Rendering is deterministic; the same anchor always produces the same text.

The rendering preserves two task-relevant properties.

Narrative order. Evidence spans are numbered by their position in the unstructured source and listed in that order. The graph stores no timestamps for the events reported by the source, so narrative order is the only ordering signal carried by the subgraph and is represented explicitly.

Withheld detail. Internal identifiers, ingest timestamps and the embeddings are never rendered.

4 Experiment Design

We evaluate ProCG using a controlled reconstruction task in which activity sequences are missing from structured event data but described in unstructured communication. Figure 7 summarizes the pipeline from the original log to scored predictions. Reconstruction selects labels from a catalog of activity types; for each, we use the textual description of what the activity *means* provided by [4], but not information about where it must occur in the sequence.

Nodes:	Relationships:
E1 EvidenceNode("the application is now cleared to move forward", this email, span=1)	A1 is made_decision D1
E2 EvidenceNode("the next step is to prepare an offer", this email, span=2)	A1 is produced_evidence E1
AC1 ActivityCluster(no details)	A1 is produced_evidence E2
P1 ProcessInstance(Bank loan application)	AC1 is has_decision D1
D1 DecisionNode(approve: Cleared to move forward decision_type=approve outcome_status=approved confidence=0.63 "...")	AC1 is part_of P1
R1 RationaleNode("Nathan states the application is now cleared to move forward [...] and adds that the next step is to prepare an offer.")	AC1 is produced E1
A1 Actor(nathan_russell actor_type=Person)	AC1 is produced E2
	D1 is acted_on AC1
	D1 is justified_by R1
	D1 is supported_by E1
	E1 is explained_by R1

Fig. 5. An example of a subgraph as the consumer receives it. The two evidence spans are numbered in the order the sender narrates them. The activity cluster carries no name, so the subgraph shows where the activities sit without naming them. Only E1 supports the decision; E2 reports a step that has not happened yet. Node text is elided at [...].

The source log is the BPI Challenge 2017 loan-application event log [13], represented as a case-centric event log [9]. We use the transformation method of Rebmman et al. [11] to convert the event log to OCEL format. Traces are grouped by the application object type (31,509 traces). Preprocessing retains only events with `lifecycle=complete` and replaces `User_*` resource identifiers with reproducible names.

We then mine contiguous activity n -grams of size 1 through 5, retaining the 20 most frequent patterns for each size (100 patterns in total). Sampling allocates a disjoint quota of traces to each pattern: a trace is used for one pattern only. The experiment draws 50 traces per pattern, giving 5,000 hidden occurrences (1,000 per gram size).

For each sampled occurrence, the n consecutive events are replaced by one placeholder email event. The remaining timeline stays observed; the dropped event sequence is the ground truth. A language model then writes one email from the hidden events, with the predecessor and successor events around the placeholder included as prompt context. The gold activity labels are not shown during reconstruction.

Characteristics of hidden emails. The longer the hidden n -gram, the longer and denser the email that stands in for it: the median body grows from 203 characters at $n = 1$ to 433 at $n = 5$. Figure 6 shows the two largest emails of the $n = 5$ set. Both hide the same pattern, `W_Complete application → A_Complete → A_Validating → O_Returned → A_Incomplete`, and both report five steps that were carried out by two other people, across three dates, with the sender narrating rather than acting. Neither email names an activity: the generator reports what happened in the wording a colleague would use, which is what makes the catalog’s near-synonymous entries hard to separate. The two examples also show that sequence length alone does not determine the outcome. In (a) every setting that sees the email or the ProCG recovers the five labels in order. In (b) the sender’s own follow-up sentence reads like a further step, and the three non-enriched text-aware settings each insert a spurious `W_Call incomplete files` before the final label; an over-generation¹ in the sense defined above; while both ProCG + enrichment settings are exact.

Reconstruction predicts the ordered activities of each hidden email. Every configuration sees the remaining OCEL timeline *before and after* the hidden span; the six settings differ only in extra context:

- *OCEL only*: timeline, no email, no graph; baseline
- *OCEL + email*: timeline plus the generated email; baseline

¹Over-generation is where the LLM generate a sequence of activities that are longer than the ground truth

(a) **Application_1577074476** recovered by every experiment setting that sees the email or the ProCG

From: terri_lewis
Subject: Update on Application_1577074476

Hi manager,

For Application_1577074476, theodore_lopez gathered the remaining details for the application so it could move forward. After that, with the offer already out, it was placed in waiting for the signed paperwork and supporting documents by theodore_lopez.

On 21 September, with caroline_schneider handling the next stage, it was put under document review. Around the same time, the offer was received back for the application by caroline_schneider.

By 23 September, when I checked the status, it was marked as still needing missing or corrected documents. I'll keep an eye on it and follow up if the paperwork does not come in promptly.

Thanks.

(b) **Application_1465506786** only the ProCG + enrichment settings are exact

From: michelle_wright
Subject: Update on Application_1465506786

Hi manager,

For Application_1465506786, francisco_small gathered the remaining details for the application, and after that it is now waiting for the signed offer and supporting documents from the customer through francisco_small. On 22 January, with elizabeth_yoder handling the file, it was placed under document review, and at the same time the offer was received back on the application by elizabeth_yoder. After I followed up on 26 January, it was moved back to the customer for missing or corrected documents. I'll keep watching for the updated paperwork so the case can move forward again.

Fig. 6. The two largest hidden emails at $n = 5$ (672 and 611 characters), both standing in for $W_Complete \rightarrow A_Complete \rightarrow A_Validating \rightarrow O_Returned \rightarrow A_Incomplete$. Each reports five steps performed by two other actors over three dates, and neither uses an activity name. In (a) the email, ProCG, combined and both ProCG + enrichment settings are exact, while OCEL only returns $A_Validating$ alone. In (b) “After I followed up on 26 January” reads as a sixth step: the email, ProCG and combined settings insert $W_Call_incomplete$ files before $A_Incomplete$; both ProCG + enrichment settings are exact. Case identifiers are abbreviated; sampling tokens are omitted.

- *OCEL + ProCG*: plus the local ProCG subgraph;
- *OCEL + email + ProCG*: email and local ProCG;
- *OCEL + ProCG + enrichment*: the local ProCG, enriched with similar decision and rationale nodes retrieved from other process instances;
- *OCEL + email + ProCG + enrichment*: both, with the same enrichment loop.

Email generation, ProCG construction, and reconstruction in all six settings use GPT-5.4.

We treat OCEL only and OCEL + email as baselines. The text-aware settings without enrichment use only local context and receive no similar decisions from other process instances. The enrichment settings additionally retrieve the decision and rationale nodes most similar to those of the case at hand from other ProCG process instances.

We report exact ordered match (all-or-nothing on the full n -gram), multiset F1 (labels, order ignored), and length-normalized Levenshtein distance (lower is better).

$$\text{lev}_{a,b}(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} \text{lev}_{a,b}(i-1, j) + 1 \\ \text{lev}_{a,b}(i, j-1) + 1 \\ \text{lev}_{a,b}(i-1, j-1) + \mathbb{I}_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

Disagreements are grouped into ordering, over-generation, under-generation², and substitution.

²Under-generation is when the LLM generate a sequence of activities that are shorter than the ground truth

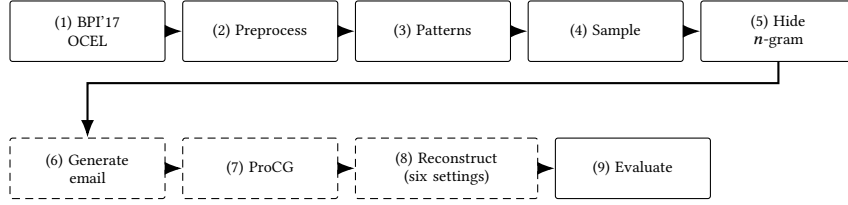


Fig. 7. From the original log to scored predictions. Solid boxes are deterministic; dashed boxes invoke an LLM. Reconstruction is run in six input configurations on the same hidden emails.

Table 2. Overall reconstruction of 5,000 hidden activity sequences. Multiset F1 and normalized edit distance report mean $\pm$ standard deviation. Best values are bold; lower normalized edit distance is better.

Configuration	Exact Match	Multiset F1	Norm. edit $\downarrow$
OCEL only (baseline)	0.024	0.141 ± 0.270	0.894 ± 0.227
OCEL + email (baseline)	0.740	0.914 ± 0.207	0.116 ± 0.236
OCEL + ProCG	0.768	0.921 ± 0.197	0.100 ± 0.221
OCEL + email + ProCG	0.770	0.923 ± 0.194	0.099 ± 0.220
OCEL + ProCG + enrichment	0.816	0.944 ± 0.164	0.071 ± 0.183
OCEL + email + ProCG + enrichment	0.821	0.948 ± 0.154	0.068 ± 0.176

5 Results

Table 2 summarizes reconstruction over 5,000 hidden activity sequences. With OCEL alone, exact match is 0.024, indicating that the surrounding observed event data are insufficient to recover the missing sequence. Adding the synthetic email raises exact match to 0.740. Exact ordered match requires the full predicted activity sequence, including order, to match the ground truth; if $A \rightarrow B$ and $B \rightarrow A$ both occur in the log, a prediction with the correct labels in the opposite order is therefore an exact mismatch, although multiset F1 still credits the recovered labels. Replacing the email with the local ProCG raises exact match to 0.768, a 2.8-percentage-point gain, while OCEL+email+ProCG reaches 0.770. This comparison indicates that the ProCG retains most information from the source email that is relevant to reconstruction while presenting it in a form that is more useful for the task. Enrichment yields the largest additional improvement: 0.816 without the email and 0.821 with it. The latter configuration also has the highest multiset F1 (0.948) and lowest normalized edit distance (0.068).

Table 3 breaks performance down by hidden sequence length. Enriched ProCG settings have the highest exact match for $n = 1-4$; at $n = 5$, the local ProCG is highest at 0.690. Email-only has the highest multiset F1 at $n = 5$ (0.944), but all five text-aware settings fall within 0.003 (0.941–0.944), while exact match ranges from 0.602 to 0.690.

The difference follows from the stricter exact-match criterion: one substitution or ordering error makes the full sequence incorrect, whereas multiset F1 still credits correctly recovered labels. The tightly clustered F1 values at $n = 5$ indicate similar recovery of activity sets, with larger differences in complete sequence reconstruction.

Table 4 separates these gains by error type. Relative to OCEL+email, OCEL+ProCG reduces ordering errors from 2.4% to 0.3%, substitutions from 11.8% to 9.6%, and under-generation from 5.4% to 3.7%, while over-generation increases from 6.4% to 9.4%. The net effect is the 2.8 percent gain in the exact match. Fig. 6b illustrates one source of over-generation: contextual or prospective language in an evidence span can resemble an additional activity.

Relative to local ProCG, enrichment gains 4.8 percentage points in exact match through reductions in over-generation (3.5 points), substitutions (2.1 points), and ordering errors (0.1 points), offset by a 0.9-point increase in under-generation.

Table 3. Reconstruction by hidden-pattern size n (5,000-occurrence run; 1,000 emails per column). Upper panel: exact ordered match. Lower panel: multiset F1 (means; standard deviations are reported for the pooled run in Table 2). Best value per column and panel in bold.

Configuration	$n=1$	$n=2$	$n=3$	$n=4$	$n=5$
<i>Exact ordered match</i>					
OCEL only (baseline)	0.076	0.023	0.006	0.007	0.008
OCEL + email (baseline)	0.752	0.712	0.793	0.839	0.602
OCEL + ProCG	0.730	0.762	0.815	0.844	0.690
OCEL + email + ProCG	0.741	0.760	0.819	0.848	0.682
OCEL + ProCG + enrichment	0.832	0.851	0.856	0.860	0.680
OCEL + email + ProCG + enrichment	0.840	0.857	0.872	0.859	0.676
<i>Multiset F1</i>					
OCEL only (baseline)	0.093	0.107	0.133	0.178	0.191
OCEL + email (baseline)	0.819	0.891	0.946	0.972	0.944
OCEL + ProCG	0.825	0.914	0.953	0.975	0.941
OCEL + email + ProCG	0.831	0.915	0.953	0.976	0.941
OCEL + ProCG + enrichment	0.886	0.953	0.965	0.973	0.942
OCEL + email + ProCG + enrichment	0.897	0.956	0.969	0.974	0.942

Table 4. Share of 5,000 scored emails per discrepancy category (rows sum to ~100%). Exact higher-better; other columns lower-better. Best value per column in bold.

Configuration	Exact	Ord.	Over	Under	Subst.
OCEL only (baseline)	2.4%	0.1%	1.0%	18.2%	78.3%
OCEL + email (baseline)	74.0%	2.4%	6.4%	5.4%	11.8%
OCEL + ProCG	76.8%	0.3%	9.4%	3.7%	9.6%
OCEL + email + ProCG	77.0%	0.4%	9.3%	3.9%	9.2%
OCEL + ProCG + enrichment	81.6%	0.2%	5.9%	4.6%	7.5%
OCEL + email + ProCG + enrichment	82.1%	0.2%	5.8%	4.5%	7.1%

Because ordering errors are already near the floor, this error profile suggests that enrichment contributes mainly to activity-label grounding rather than sequence ordering. Retrieved cases provide related process context in which semantically similar decisions and rationales are linked to observed activity labels; Sect. 5.1 gives a concrete example.

The large multiset-F1 standard deviations reflect a mix of exact recoveries (score 1.0) and a tail of substitutions.

5.1 Worked example: cross-instance enrichment

Application 1505059688 contains one hidden email. We denote its position by **[email]**; the observed timeline is

... $\rightarrow$ O_Created $\rightarrow$ O_Sent $\rightarrow$ **[email]** $\rightarrow$ O_Accepted, A_Pending

and the five activities behind **[email]** are W_Complete $\rightarrow$ A_Complete $\rightarrow$ A_Validating $\rightarrow$ O_Returned $\rightarrow$ A_Incomplete. The local ProCG returns all five activities but inserts a sixth, W_Validate, before the final activity, producing one extra activity with confidence 0.87.

The ambiguity comes from span 5:

“After I reviewed it on 2016-11-09, it was marked as needing additional documents from the customer.”

The span states that the case was marked as needing documents, corresponding to A_Incomplete. It also mentions a review by the sender, which matches the meaning of W_Validate. The local ProCG therefore predicts both activities; its stated reason is that the spans narrate *“receipt and review of returned documents, and then marking the case incomplete.”*

Table 5. The two process instances opened for Application 1505059688. **[email]** marks that case’s own hidden email. The middle column is what that case hides: the tool never shows it, and it is given here so the reader can check that the labels in the last column are observed event data rather than another case’s answer. A leading or trailing ... means events are omitted.

Process instance	What it hides (withheld)	Its observed log, with the hidden email in place
1173116422	W_Validate → A_Incomplete	... A_Validating → O_Returned → A_Incomplete ... [email] ...
1775664277	A_Complete → A_Validating → O_Returned → A_Incomplete	... W_Complete → [email] → A_Validating → A_Incomplete → O_Accepted ...

Retrieved context. The LLM queried the target decision and rationale for similar nodes (Sect. 3.3.2), returning ten candidates in total. It then opened two associated process instances (Sect. 3.3.3): Applications 1173116422 and 1775664277, ranked third and fourth among the similar rationales. It opened neither of the two nearest rationales nor any of the five similar decisions. Table 5 shows the corresponding logs.

Evidence from the retrieved logs. Both logs reach A_Incomplete, and neither places W_Validate immediately before it. Application 1173116422 has O_Returned → A_Incomplete as consecutive observed events; the exact pair that the local ProCG split apart; and 1775664277 has A_Validating → A_Incomplete the same way. Both examples show that A_Incomplete can be reached without an intervening validation step in the observed log.

With this additional context, the LLM no longer treats the review in span 5 as a sixth event. It returns all five gold activities, reasoning that the spans conclude with the case being marked incomplete, and that the final span describes only a future follow-up action rather than something that already happened.

Structural relation of the retrieved cases. Eight of the ten candidates returned by the two queries share an actor with the target case; the two that do not have the two lowest structural scores. Actor identities are the only joins between per-email subgraphs Fig. 4, so the structural vector places a node among the cases its actors touched and cross-process retrieval travels along those joins. Comparable cases are therefore often other applications handled by the same actors. They remain distinct applications, and the system explicitly rejects another sample from the case under reconstruction, so the retrieved information comes from a different case rather than revealing the answer.

Limit of the local ProCG. Both settings read the same six spans; only the retrieved process instances differ. A span that mentions two actions need not correspond to two event occurrences, and the target case’s text and timeline do not resolve that ambiguity. The retrieved logs provide an additional distinction.

6 Conclusion

We introduced the Process Context Graph (ProCG), a knowledge-graph-based intermediate representation that connects process-relevant information extracted from unstructured sources to structured event data. ProCG preserves actors, evidence, decisions, rationales, and their relationships rather than mapping text directly to events and attributes, and supports both local context access and cross-process retrieval.

In a controlled reconstruction experiment on 5,000 hidden activity sequences from the BPI Challenge 2017 log, the context-graph representation outperformed the raw email in the local setting, while combining the two representations produced only a small additional improvement. The best performance was obtained with ProCG with enrichment combined with the email. The results indicate that an explicitly structured textual context can make process-relevant information more useful for reconstructing behavior missing from structured event data.

The evaluation uses generated emails and a single process domain under controlled conditions. Future work should examine real-world unstructured sources and a broader range of process-mining tasks.

Code availability. The implementation is publicly available at <https://github.com/clearsky-ai/process-context-graph>.

References

- [1] Alessandro Berti, Humam Kourani, and Wil M. P. van der Aalst. 2025. PM-LLM-Benchmark: Evaluating Large Language Models on Process Mining Tasks. In *Process Mining Workshops (Lecture Notes in Business Information Processing, Vol. 533)*, Andrea Delgado and Tijs Slaats (Eds.). Springer, 610–623. doi:10.1007/978-3-031-82225-4_45
- [2] Alessandro Berti and Mahnaz Sadat Qafari. 2023. Leveraging Large Language Models (LLMs) for Process Mining. *CoRR* abs/2307.12701 (2023). arXiv:2307.12701 doi:10.48550/arXiv.2307.12701
- [3] Alina Buss, Christoph Kecht, Wolfgang Kratsch, Maximilian Röglinger, Sareh Sadeghianasl, and Moe T. Wynn. 2026. Process Mining Between the Lines: Extracting Object-Centric Event Logs from Textual Data. *Information Systems* 140 (2026), 102713. doi:10.1016/j.is.2026.102713
- [4] Raimundo Carmona, Rodrigo Cofré, Carolina Naranjo, Orlando Chuquisengo Vásquez, Jonathan Lee, Juan P. Salazar Fernández, and Michael Arias. 2017. Analysis of Loan Application Process Using Process Mining. <https://api.semanticscholar.org/CorpusID:195655623>
- [5] Haochen Chen, Syed Fahad Sultan, Yingtao Tian, Muhao Chen, and Steven Skiena. 2019. Fast and Accurate Network Embeddings via Very Sparse Random Projection. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3-7, 2019*, Wenwu Zhu, Dacheng Tao, Xueqi Cheng, Peng Cui, Elke A. Rundensteiner, David Carmel, Qi He, and Jeffrey Xu Yu (Eds.). ACM, 399–408. doi:10.1145/3357384.3357879
- [6] Vinicius Stein Dani, Marcus Dees, Henrik Leopold, Kiran Busch, Iris Beerepoot, Jan Martijn E. M. van der Werf, and Hajo A. Reijers. 2025. Event Log Extraction for Process Mining Using Large Language Models. In *Cooperative Information Systems (Lecture Notes in Computer Science, Vol. 15506)*. Springer, 56–72. doi:10.1007/978-3-031-81375-7_4
- [7] Dirk Fahland. 2022. Process Mining over Multiple Behavioral Dimensions with Event Knowledge Graphs. In *Process Mining Handbook*, Wil M. P. van der Aalst and Josep Carmona (Eds.). Lecture Notes in Business Information Processing, Vol. 448. Springer, 274–319. doi:10.1007/978-3-031-08848-3_9
- [8] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a Graph: Encoding Graphs for Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=luXR1CCrSi>
- [9] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M. P. van der Aalst. 2021. OCEL: A Standard for Object-Centric Event Logs. In *New Trends in Database and Information Systems - ADBIS 2021 Short Papers, Doctoral Consortium and Workshops: DOING, SIMPDA, MADEISD, MegaData, CAoNS, Tartu, Estonia, August 24-26, 2021, Proceedings (Communications in Computer and Information Science, Vol. 1450)*, Ladjel Bellatreche, Marlon Dumas, Panagiotis Karras, Raimundas Matulevicius, Ahmed Awad, Matthias Weidlich, Mirjana Ivanovic, and Olaf Hartig (Eds.). Springer, 169–175. doi:10.1007/978-3-030-85082-1_16
- [10] Eva L. Klijn, Felix Mannhardt, and Dirk Fahland. 2023. Aggregating Event Knowledge Graphs for Task Analysis. In *Process Mining Workshops (Lecture Notes in Business Information Processing, Vol. 468)*, Marco Montali, Arik Senderovich, and Matthias Weidlich (Eds.). Springer, 493–505. doi:10.1007/978-3-031-27815-0_36
- [11] Adrian Rebmann, Jana-Rebecca Rehse, and Han van der Aa. 2022. Uncovering Object-Centric Data in Classical Event Logs for the Automated Transformation from XES to OCEL. In *Business Process Management - 20th International Conference, BPM 2022, Münster, Germany, September 11-16, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13420)*, Claudio Di Ciccio, Remco M. Dijkman, Adela del-Rio-Ortega, and Stefanie Rinderle-Ma (Eds.). Springer, 379–396. doi:10.1007/978-3-031-16103-2_25
- [12] Adrian Rebmann, Fabian David Schmidt, Goran Glavaš, and Han van der Aa. 2025. On the Potential of Large Language Models to Solve Semantics-Aware Process Mining Tasks. *Process Science* 2 (2025), 10. doi:10.1007/s44311-025-00019-3
- [13] Boudewijn van Dongen. 2017. BPI Challenge 2017. doi:10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b
- [14] Chengjin Xu, Muzhi Li, Cehao Yang, Xuhui Jiang, Luminyuan Tang, Yiyan Qi, and Jian Guo. 2024. Context Graph. *CoRR* abs/2406.11160 (2024). arXiv:2406.11160 doi:10.48550/ARXIV.2406.11160